

QUIZ - OBJECT ORIENTED PROGRAMMING

Case Study: "Bengkel Maju" Automotive Workshop System

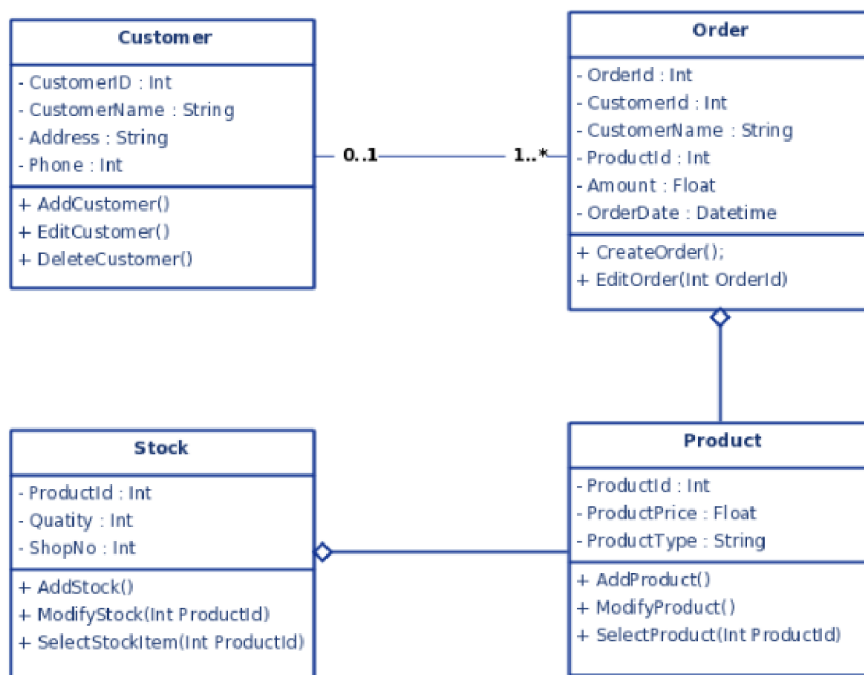
Field	Description
Name	Mafaza Husnadani
Student ID	254107060130
Class / No.	SIB-2G / 14
Course	Practicum PBO

The quiz consists of two questions. Question 1 asks for the given class diagram to be corrected, explained, and implemented in Java. Question 2 asks for a new class diagram and Java program for the "Bengkel Maju" workshop. This document is divided so that PART 1 and PART 2 answer Question 1, while PART 3, PART 4, and PART 5 answer Question 2.

PART 1 - QUESTION 1: Correcting the Given Class Diagram

Examine the following class diagram. Correct the class diagram according to proper UML notation and justify the corrections you make. After revising the diagram, explain the relationships between the classes and the UML notations used. Finally, implement the corrected class diagram as a Java program.

Class Diagram for Order Processing System



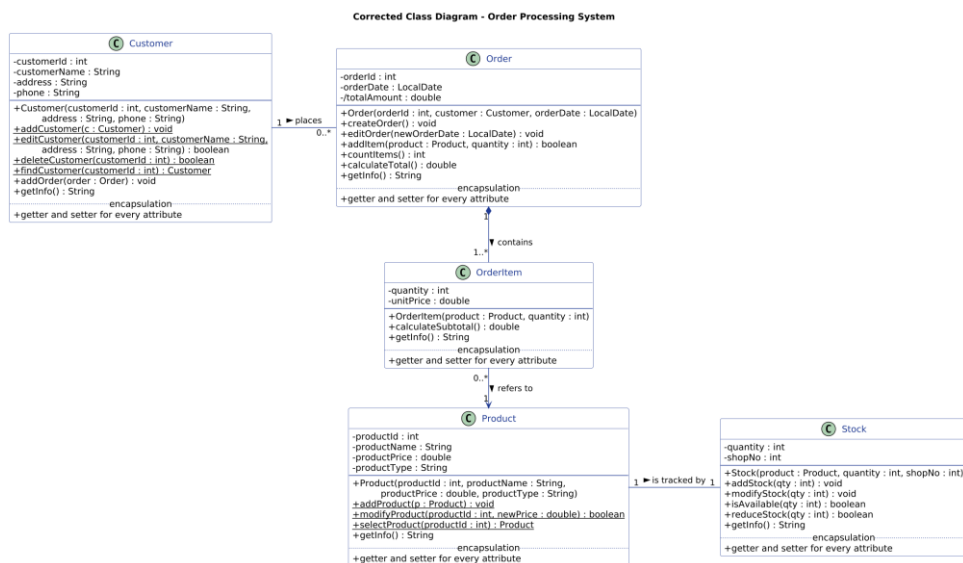
1. List of Corrections and Their Justifications

No.	Problem Found in the Original Diagram	Correction Applied	Justification
1	Attribute and operation names start with a capital	Renamed to lower camelCase: customerId, addCustomer().	UML naming convention states that class names use UpperCamelCase while attributes and operations use

No.	Problem Found in the Original Diagram	Correction Applied	Justification
	letter (CustomerID, AddCustomer()).		lowerCamelCase. This also matches Java naming rules, so the diagram maps directly to code.
2	Operations have no return type and no parameters, e.g. + AddCustomer().	Written in full UML form: + addCustomer(c : Customer) : void.	The UML operation signature is visibility name(parameter : Type) : ReturnType. Without it the reader cannot tell what the operation needs or produces, and the diagram is not implementable.
3	The operation + CreateOrder(); ends with a semicolon.	Semicolon removed.	A semicolon is Java syntax, not UML notation. A class diagram must not contain language-specific statement terminators.
4	Phone : Int	phone : String	A phone number is not a quantity. It may begin with 0 and contain +, -, or spaces, and no arithmetic is ever performed on it, so String is the correct type.
5	Amount : Float and ProductPrice : Float	totalAmount : double and productPrice : double (totalAmount marked as derived).	Float loses precision for monetary values. In addition, totalAmount is computed from the order items, so in UML it is written as a derived attribute (/totalAmount) rather than a stored one.
6	OrderDate : Datetime	orderDate : LocalDate	Datetime is a database type, not a UML/Java type. The diagram must use the type used by the target platform.
7	Typing error: Quatity : Int in class Stock.	quantity : int	A misspelled attribute name would be carried into the source code and reduces the readability of the model.
8	Order stores CustomerId and CustomerName; Order and Stock both store ProductId.	These foreign-key attributes were removed and replaced by associations.	Storing an identifier of another class is relational-database thinking. In UML/OOP the association itself provides navigation (order.getCustomer().getName()). Duplicating CustomerName also creates redundant data that can become inconsistent.
9	Multiplicity Customer 0..1 -- 1..* Order.	Corrected to Customer 1 -- 0..* Order.	Reading the original: "an order may belong to zero customers", which is impossible - an order always has exactly one owner. Conversely a newly registered customer may still have no order, so the Order end must be 0..*.
10	The associations have no name and no reading direction.	Named associations added: places, contains, refers to, is tracked by, with a solid triangle showing the reading direction.	UML recommends labelling an association with a verb phrase so the relationship can be read as a sentence: "a Customer places an Order".
11	The relationships Order-Product and Stock-Product have no multiplicity at all.	Multiplicity added at both ends of every relationship.	Multiplicity is mandatory information in a class diagram; without it the model does not state how many objects participate.
12	Order is connected directly to Product with an aggregation diamond.	An association class OrderItem was inserted between Order and Product, using composition from	An order line needs its own data (quantity, unit price) that belongs neither to Order nor to Product. Order items also cannot exist without their order, so the Order-OrderItem

No.	Problem Found in the Original Diagram	Correction Applied	Justification
		Order and a directed association to Product.	relationship is a composition (filled diamond), while Product exists independently and is only referenced.
13	The aggregation diamond between Stock and Product is placed on the Stock side.	Replaced by a 1-to-1 association "Product is tracked by Stock".	In UML the diamond must sit at the "whole" end. Stock is not a whole that consists of products; it is a record describing the availability of one product, so an aggregation here is conceptually reversed.
14	Class Stock contains the operation SelectStockItem(Int ProductId), which duplicates the responsibility of Product.	Replaced by isAvailable(qty : int) : boolean.	Single Responsibility Principle: selecting a product belongs to Product, while checking availability belongs to Stock.
15	addCustomer(), editCustomer(), deleteCustomer(), addProduct(), modifyProduct() and selectProduct() are drawn as ordinary instance operations.	Marked as class-level (static) operations, shown underlined in UML.	These operations manage the whole collection of customers or products, not a single object. In UML a class-scope member is underlined, and in Java it becomes a static method.

2. The Corrected Class Diagram



3. Explanation of the Relationships and UML Notation Used

Customer "1" -- "0..*" Order (association, "places"). A plain solid line is an association: two independent classes that simply know each other. One customer may place many orders, and every order belongs to exactly one customer. The multiplicity 1 on the Customer end and 0..* on the Order end expresses this.

Order "1" *-- "1..*" OrderItem (composition, "contains"). The filled diamond on the Order end marks a composition: OrderItem is a part whose life cycle is fully controlled by the whole. If the order is deleted, its items are deleted with it, and an item can never be shared by two orders. An order must have at least one item, hence 1..*.

OrderItem "0..*" --> "1" Product (directed association, "refers to"). The open arrowhead shows navigability: an order item knows its product, but a product does not keep a list of the order items that reference it. A product exists in the catalogue independently of any order, so this is an ordinary association and not a composition.

Product "1" -- "1" Stock (association, "is tracked by"). Each product has exactly one stock record and each stock record describes exactly one product, giving a one-to-one association.

4. Summary of the UML Notation Symbols

Notation	Meaning	Where it is used in the diagram
Rectangle with three compartments	Class: name, attributes, operations	All five classes
- (minus)	private visibility	All attributes (encapsulation)
+ (plus)	public visibility	All operations
name : Type	Attribute with its data type	customerId : int
name(param : Type) : ReturnType	Operation signature	addItem(product : Product, quantity : int) : boolean
/name	Derived attribute (computed, not stored)	/totalAmount : double
Underlined member	Class-scope (static) operation	addCustomer(c : Customer) : void
Plain solid line	Association	Customer - Order, Product - Stock
Line with open arrowhead	Directed (navigable) association	OrderItem -> Product
Filled diamond	Composition (strong "part-of")	Order - OrderItem
Hollow diamond	Aggregation (weak "part-of")	Used in PART 3: Customer - Vehicle
1, 0..*, 1..*	Multiplicity	At both ends of every relationship
Verb phrase + solid triangle	Association name and reading direction	places, contains, refers to

PART 2 - QUESTION 1: Java Implementation and Output

The corrected class diagram above is implemented in six Java files, stored in the folder Quiz/Soal1 and declared as package Soal1. The answers to the two questions use several class names in common (Customer, Order), so each one is placed in its own package to keep the two programs independent. From the folder Quiz the program is compiled with `javac Soal1/*.java` and executed with `java Soal1.OrderProcessingDemo14`.

File	Role in the diagram
Customer14.java	Class Customer, including the static operations <code>addCustomer()</code> , <code>editCustomer()</code> , <code>deleteCustomer()</code> and <code>findCustomer()</code>
Order14.java	Class Order, owns its OrderItem objects (composition) and computes the derived <code>totalAmount</code>
OrderItem14.java	Class OrderItem, holds quantity and unitPrice and calculates the subtotal
Product14.java	Class Product, including the static operations <code>addProduct()</code> , <code>modifyProduct()</code> and <code>selectProduct()</code>
Stock14.java	Class Stock, one-to-one with Product, checks and updates availability
OrderProcessingDemo14.java	Main program that creates the objects and prints all information

1. Customer14.java

Customer14.java U X

Quiz > Soal1 > Customer14.java > Customer14 > editCustomer(int, String, String, String)

```
1 package Soal1;
2
3 import java.util.ArrayList;
4
5 public class Customer14 {
6     private static ArrayList<Customer14> customerList = new ArrayList<Customer14>();
7
8     private int customerId;
9     private String customerName;
10    private String address;
11    private String phone;
12    private ArrayList<Order14> orders;
13
14    public Customer14(int customerId, String customerName, String address,
15        String phone) {
16        this.customerId = customerId;
17        this.customerName = customerName;
18        this.address = address;
19        this.phone = phone;
20        this.orders = new ArrayList<Order14>();
21    }
22
23    public int getCustomerId() {
24        return customerId;
25    }
26
27    public void setCustomerId(int customerId) {
28        this.customerId = customerId;
29    }
30
31    public String getCustomerName() {
32        return customerName;
33    }
34
35    public void setCustomerName(String customerName) {
36        this.customerName = customerName;
37    }
38}
```

Customer14.java U X

Quiz > Soal1 > Customer14.java > Customer14 > editCustomer(int, String, String, String)

```
5 public class Customer14 {
35    public void setCustomerName(String customerName) {
36        this.customerName = customerName;
37    }
38
39    public String getAddress() {
40        return address;
41    }
42
43    public void setAddress(String address) {
44        this.address = address;
45    }
46
47    public String getPhone() {
48        return phone;
49    }
50
51    public void setPhone(String phone) {
52        this.phone = phone;
53    }
54
55    public ArrayList<Order14> getOrders() {
56        return orders;
57    }
58
59    public void addOrder(Order14 order) {
60        if (order != null && !orders.contains(order)) {
61            orders.add(order);
62        }
63    }
64
65    public static void addCustomer(Customer14 c) {
66        if (c != null && findCustomer(c.getCustomerId()) == null) {
67            customerList.add(c);
68        }
69    }
}
```

```

Customer14.java U X
Quiz > Soal1 > Customer14.java > Customer14 > editCustomer(int, String, String, String)
5 public class Customer14 {
71     public static boolean editCustomer(int customerId, String customerName,
72         String address, String phone) {
73         Customer14 c = findCustomer(customerId);
74
75         if (c == null) {
76             return false;
77         }
78
79         c.setCustomerName(customerName);
80         c.setAddress(address);
81         c.setPhone(phone);
82         return true;
83     }
84
85     public static boolean deleteCustomer(int customerId) {
86         Customer14 c = findCustomer(customerId);
87
88         if (c == null) {
89             return false;
90         }
91
92         customerList.remove(c);
93         return true;
94     }
95
96     public static Customer14 findCustomer(int customerId) {
97         for (Customer14 c : customerList) {
98             if (c.getCustomerId() == customerId) {
99                 return c;
100             }
101         }
102
103         return null;
104     }
105
106     public static ArrayList<Customer14> getCustomerList() {
107         return customerList;
108     }
109
110     public String getInfo() {
111         return customerName + " (ID " + customerId + "), " + address
112             + ", telp " + phone;
113     }
114 }
115

```

2. Order14.java

```

Order14.java U X
Quiz > Soal1 > Order14.java > Order14 > Order14(int, Customer14, LocalDate)
1  package Soal1;
2
3  import java.time.LocalDate;
4  import java.util.ArrayList;
5
6  public class Order14 {
7      private int orderId;
8      private LocalDate orderDate;
9      private Customer14 customer;
10     private ArrayList<OrderItem14> items;
11
12     public Order14(int orderId, Customer14 customer, LocalDate orderDate) {
13         this.orderId = orderId;
14         this.customer = customer;
15         this.orderDate = orderDate;
16         this.items = new ArrayList<OrderItem14>();
17     }
18
19     public int getOrderId() {
20         return orderId;
21     }
22
23     public void setOrderId(int orderId) {
24         this.orderId = orderId;
25     }
26
27     public LocalDate getOrderDate() {
28         return orderDate;
29     }
30
31     public void setOrderDate(LocalDate orderDate) {
32         this.orderDate = orderDate;
33     }
34
35     public Customer14 getCustomer() {
36         return customer;
37     }

```

```

Order14.java U X
Quiz > Soal1 > Order14.java > Order14 > Order14(int, Customer14, LocalDate)
6  public class Order14 {
35     public Customer14 getCustomer() {
36         return customer;
37     }
38
39     public void setCustomer(Customer14 customer) {
40         this.customer = customer;
41     }
42
43     public ArrayList<OrderItem14> getItems() {
44         return items;
45     }
46
47     public void createOrder() {
48         if (customer != null) {
49             customer.addOrder(this);
50         }
51     }
52
53     public void editOrder(LocalDate newOrderDate) {
54         if (newOrderDate != null) {
55             this.orderDate = newOrderDate;
56         }
57     }
58
59     public boolean addItem(Product14 product, int quantity) {
60         if (product == null || quantity <= 0) {
61             return false;
62         }
63
64         Stock14 stock = product.getStock();
65
66         if (stock == null || !stock.isAvailable(quantity)) {
67             return false;
68         }
69     }

```

```

Order14.java
Quiz > Soal1 > Order14.java > Order14 > Order14(int, Customer14, LocalDate)
6   public class Order14 {
59   public boolean addItem(Product14 product, int quantity) {
69       stock.reduceStock(quantity);
70       items.add(new OrderItem14(product, quantity));
71       return true;
72   }
73   }
74
75   public int countItems() {
76       return items.size();
77   }
78
79   public double calculateTotal() {
80       double total = 0;
81
82       for (OrderItem14 item : items) {
83           total += item.calculateSubtotal();
84       }
85
86       return total;
87   }
88
89   public String getInfo() {
90       String info = "";
91       info += "No. Order    : " + orderId + "\n";
92       info += "Tanggal      : " + orderDate + "\n";
93       info += "Pelanggan   : " + customer.getInfo() + "\n";
94       info += "Daftar Item : \n";
95
96       for (OrderItem14 item : items) {
97           info += "    - " + item.getInfo() + "\n";
98       }
99
100      info += "Jumlah Item : " + countItems() + "\n";
101      info += "TOTAL      : Rp" + (long) calculateTotal() + "\n";
102
103      return info;

```

3. OrderItem14.java

```

OrderItem14.java U X
Quiz > Soal1 > OrderItem14.java > {} Soal1
1  package Soal1;
2
3  public class OrderItem14 {
4      private int quantity;
5      private double unitPrice;
6      private Product14 product;
7
8      public OrderItem14(Product14 product, int quantity) {
9          this.product = product;
10         this.quantity = quantity;
11         this.unitPrice = product.getProductPrice();
12     }
13
14     public int getQuantity() {
15         return quantity;
16     }
17
18     public void setQuantity(int quantity) {
19         if (quantity > 0) {
20             this.quantity = quantity;
21         }
22     }
23
24     public double getUnitPrice() {
25         return unitPrice;
26     }
27
28     public void setUnitPrice(double unitPrice) {
29         if (unitPrice >= 0) {
30             this.unitPrice = unitPrice;
31         }
32     }
33
34     public Product14 getProduct() {
35         return product;
36     }
37
38     public void setProduct(Product14 product) {
39         this.product = product;
40     }
41
42     public double calculateSubtotal() {
43         return unitPrice * quantity;
44     }
45
46     public String getInfo() {
47         return product.getProductName() + " x" + quantity + " @Rp"
48             + (long) unitPrice + " = Rp" + (long) calculateSubtotal();
49     }
50 }
51

```

4. Product14.java

Product14.java U X

Quiz > Soal1 > Product14.java > {} Soal1

```
1 package Soal1;
2
3 import java.util.ArrayList;
4
5 public class Product14 {
6     private static ArrayList<Product14> productList = new ArrayList<Product14>();
7
8     private int productId;
9     private String productName;
10    private double productPrice;
11    private String productType;
12    private Stock14 stock;
13
14    public Product14(int productId, String productName, double productPrice,
15        String productType) {
16        this.productId = productId;
17        this.productName = productName;
18        this.productPrice = productPrice;
19        this.productType = productType;
20    }
21
22    public int getProductId() {
23        return productId;
24    }
25
26    public void setProductId(int productId) {
27        this.productId = productId;
28    }
29
30    public String getProductName() {
31        return productName;
32    }
33
34    public void setProductName(String productName) {
35        this.productName = productName;
36    }
37
```

Product14.java U X

Quiz > Soal1 > Product14.java > {} Soal1

```
5 public class Product14 {
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38    public double getProductPrice() {
39        return productPrice;
40    }
41
42    public void setProductPrice(double productPrice) {
43        if (productPrice >= 0) {
44            this.productPrice = productPrice;
45        }
46    }
47
48    public String getProductType() {
49        return productType;
50    }
51
52    public void setProductType(String productType) {
53        this.productType = productType;
54    }
55
56    public Stock14 getStock() {
57        return stock;
58    }
59
60    public void setStock(Stock14 stock) {
61        this.stock = stock;
62    }
63
64    public static void addProduct(Product14 p) {
65        if (p != null && selectProduct(p.getProductId()) == null) {
66            productList.add(p);
67        }
68    }
69
70    public static boolean modifyProduct(int productId, double newPrice) {
71        Product14 p = selectProduct(productId);
72    }
73
```

```
Product14.java U X
Quiz > Soal1 > Product14.java > {} Soal1
5 public class Product14 {
68 }
69
70 public static boolean modifyProduct(int productId, double newPrice) {
71     Product14 p = selectProduct(productId);
72
73     if (p == null) {
74         return false;
75     }
76
77     p.setProductPrice(newPrice);
78     return true;
79 }
80
81 public static Product14 selectProduct(int productId) {
82     for (Product14 p : productList) {
83         if (p.getProductId() == productId) {
84             return p;
85         }
86     }
87
88     return null;
89 }
90
91 public static ArrayList<Product14> getProductList() {
92     return productList;
93 }
94
95 public String getInfo() {
96     return productName + " (" + productType + ") - Rp"
97         + (long) productPrice;
98 }
99 }
100
```

5. Stock14.java

Stock14.java U X

Quiz > Soal1 > Stock14.java > {} Soal1

```
1  package Soal1;
2
3  public class Stock14 {
4      private int quantity;
5      private int shopNo;
6      private Product14 product;
7
8      public Stock14(Product14 product, int quantity, int shopNo) {
9          this.product = product;
10         this.quantity = quantity;
11         this.shopNo = shopNo;
12
13         if (product != null) {
14             product.setStock(this);
15         }
16     }
17
18     public int getQuantity() {
19         return quantity;
20     }
21
22     public void setQuantity(int quantity) {
23         if (quantity >= 0) {
24             this.quantity = quantity;
25         }
26     }
27
28     public int getShopNo() {
29         return shopNo;
30     }
31
32     public void setShopNo(int shopNo) {
33         this.shopNo = shopNo;
34     }
35
36     public Product14 getProduct() {
37         return product;
38     }
39 }
```

Stock14.java U X

Quiz > Soal1 > Stock14.java > {} Soal1

```
3  public class Stock14 {
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36     public Product14 getProduct() {
37         return product;
38     }
39
40     public void setProduct(Product14 product) {
41         this.product = product;
42     }
43
44     public void addStock(int qty) {
45         if (qty > 0) {
46             this.quantity += qty;
47         }
48     }
49
50     public void modifyStock(int qty) {
51         setQuantity(qty);
52     }
53
54     public boolean isAvailable(int qty) {
55         return qty > 0 && this.quantity >= qty;
56     }
57
58     public boolean reduceStock(int qty) {
59         if (!isAvailable(qty)) {
60             return false;
61         }
62
63         this.quantity -= qty;
64         return true;
65     }
66
67     public String getInfo() {
68         return "Toko " + shopNo + " - sisa stok " + quantity + " unit";
69     }
70 }
```

6. OrderProcessingDemo14.java (Main)

```
OrderProcessingDemo14.java U X
Quiz > Soal1 > OrderProcessingDemo14.java > ...
1 package Soal1;
2
3 import java.time.LocalDate;
4
5 public class OrderProcessingDemo14 {
6     Run | Debug
7     public static void main(String[] args) {
8         // 1. Data pelanggan
9         Customer14 mafaza = new Customer14(
10             customerId: 1, customerName: "Mafaza Husnadani", address: "Jl. Soekarno Hatta, Malang", phone: "081234567890");
11         Customer14 rafi = new Customer14(
12             customerId: 2, customerName: "Rafi Ardiansyah", address: "Jl. Veteran, Malang", phone: "085612340987");
13
14         Customer14.addCustomer(mafaza);
15         Customer14.addCustomer(rafy);
16
17         // 2. Data produk dan stoknya
18         Product14 keyboard = new Product14(productId: 101, productName: "Keyboard Mekanik", productPrice: 450000, productT... "Aksesoris");
19         Product14 mouse = new Product14(productId: 102, productName: "Mouse Wireless", productPrice: 175000, productT... "Aksesoris");
20         Product14 monitor = new Product14(productId: 103, productName: "Monitor 24 inch", productPrice: 1850000, productT... "Elektronik");
21
22         Product14.addProduct(keyboard);
23         Product14.addProduct(mouse);
24         Product14.addProduct(monitor);
25
26         // Relasi 1 ke 1: setiap produk dicatat oleh satu data stok
27         new Stock14(keyboard, quantity: 10, shopNo: 1);
28         new Stock14(mouse, quantity: 25, shopNo: 1);
29         new Stock14(monitor, quantity: 3, shopNo: 2);
30
31         System.out.println(x: "=====");
32         System.out.println(x: "          DAFTAR PRODUK DAN STOK AWAL          ");
33         System.out.println(x: "=====");
34
35         for (Product14 p : Product14.getProductList()) {
36             System.out.println(p.getInfo() + " | " + p.getStock().getInfo());
37         }
38     }
39 }
```

```
OrderProcessingDemo14.java U X
Quiz > Soal1 > OrderProcessingDemo14.java > ...
5 public class OrderProcessingDemo14 {
6     public static void main(String[] args) {
7
8         System.out.println();
9
10        // 3. Order pertama
11        Order14 order1 = new Order14(orderId: 1001, mafaza, LocalDate.of(year: 2026, month: 9, dayOfMonth: 22));
12        order1.createOrder();
13        order1.addItem(keyboard, quantity: 2);
14        order1.addItem(mouse, quantity: 1);
15
16        // 4. Order kedua milik pelanggan yang sama
17        Order14 order2 = new Order14(orderId: 1002, mafaza, LocalDate.of(year: 2026, month: 9, dayOfMonth: 22));
18        order2.createOrder();
19        order2.addItem(monitor, quantity: 1);
20
21        // 5. Order milik pelanggan lain
22        Order14 order3 = new Order14(orderId: 1003, rafi, LocalDate.of(year: 2026, month: 9, dayOfMonth: 23));
23        order3.createOrder();
24        order3.addItem(mouse, quantity: 3);
25
26        // Percobaan menambah item melebihi stok yang tersedia
27        boolean gagal = order3.addItem(monitor, quantity: 10);
28        System.out.println("Menambah 10 unit Monitor (stok hanya 2) : " + gagal
29            + " (ditolak oleh Stock14.isAvailable())");
30        System.out.println();
31
32        // 6. Tampilkan seluruh order
33        System.out.println(x: "=====");
34        System.out.println(x: "          DAFTAR ORDER          ");
35        System.out.println(x: "=====");
36
37        for (Customer14 c : Customer14.getCustomerList()) {
38            System.out.println(">> Pelanggan: " + c.getCustomerName()
39                + " memiliki " + c.getOrders().size() + " order");
40            System.out.println();
41        }
42    }
43 }
```

```

OrderProcessingDemo14.java U X
Quiz > Soal1 > OrderProcessingDemo14.java > ...
5  public class OrderProcessingDemo14 {
6      public static void main(String[] args) {
67         for (Customer14 c : Customer14.getCustomerList()) {
68             System.out.println(">> Pelanggan: " + c.getCustomerName()
69                 + " memiliki " + c.getOrders().size() + " order");
70             System.out.println();
71
72             for (Order14 o : c.getOrders()) {
73                 System.out.println(o.getInfo());
74                 System.out.println(x: "-----");
75             }
76         }
77
78         // 7. Uji operasi statis dan perubahan stok
79         System.out.println(x: "=====");
80         System.out.println(x: "        UJI OPERASI CRUD DAN STOK        ");
81         System.out.println(x: "=====");
82
83         Product14.modifyProduct(productId: 102, newPrice: 165000);
84         System.out.println("Harga mouse setelah modifyProduct() : Rp"
85             + (long) mouse.getProductPrice());
86
87         Customer14.editCustomer(customerId: 2, customerName: "Rafi Ardiansyah", address: "Jl. Ijen, Malang", phone: "085612340987");
88         System.out.println("Alamat Rafi setelah editCustomer() : " + rafi.getAddress());
89
90         System.out.println(x: "Stok setelah semua order      :");
91
92         for (Product14 p : Product14.getProductList()) {
93             System.out.println("  - " + p.getProduct_name() + " : "
94                 + p.getStock().getQuantity() + " unit");
95         }
96
97         System.out.println("deleteCustomer(99) (ID tidak ada) : "
98             + Customer14.deleteCustomer(customerId: 99));
99     }
100 }
101

```

7. Program Output

```

Problems Output Debug Console Terminal Ports
Run: OrderProcessingDemo14 + - [ ] ... [C] X

PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet\bin' 'Soal1.OrderProcessingDemo14'

=====
DAFTAR PRODUK DAN STOK AWAL
=====
Keyboard Mekanik (Aksesoris) - Rp450000 | Toko 1 - sisa stok 10 unit
Mouse Wireless (Aksesoris) - Rp175000 | Toko 1 - sisa stok 25 unit
Monitor 24 inch (Elektronik) - Rp1850000 | Toko 2 - sisa stok 3 unit

Menambah 10 unit Monitor (stok hanya 2) : false (ditolak oleh Stock14.isAvailable())

=====
DAFTAR ORDER
=====
>> Pelanggan: Mafaza Husnadani memiliki 2 order

No. Order : 1001
Tanggal : 2026-09-22
Pelanggan : Mafaza Husnadani (ID 1), Jl. Soekarno Hatta, Malang, telp 081234567890
Daftar Item :
- Keyboard Mekanik x2 @Rp450000 = Rp900000
- Mouse Wireless x1 @Rp175000 = Rp175000
Jumlah Item : 2
TOTAL : Rp1075000

-----
No. Order : 1002
Tanggal : 2026-09-22
Pelanggan : Mafaza Husnadani (ID 1), Jl. Soekarno Hatta, Malang, telp 081234567890
Daftar Item :
- Monitor 24 inch x1 @Rp1850000 = Rp1850000
Jumlah Item : 1
TOTAL : Rp1850000

-----
>> Pelanggan: Rafi Ardiansyah memiliki 1 order

No. Order : 1003
Tanggal : 2026-09-23

>> Pelanggan: Rafi Ardiansyah memiliki 1 order

No. Order : 1003
Tanggal : 2026-09-23
Pelanggan : Rafi Ardiansyah (ID 2), Jl. Veteran, Malang, telp 085612340987
Daftar Item :
- Mouse Wireless x3 @Rp175000 = Rp525000
Jumlah Item : 1
TOTAL : Rp525000

-----
=====
UJI OPERASI CRUD DAN STOK
=====
Harga mouse setelah modifyProduct() : Rp165000
Alamat Rafi setelah editCustomer() : Jl. Ijen, Malang
Stok setelah semua order :
- Keyboard Mekanik : 8 unit
- Mouse Wireless : 21 unit
- Monitor 24 inch : 2 unit
deleteCustomer(99) (ID tidak ada) : false
PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet>

```

Verification of the calculation:

Order	Customer	Items	Total
1001	Mafaza Husnadani	Keyboard Mekanik x2, Mouse Wireless x1	Rp1,075,000
1002	Mafaza Husnadani	Monitor 24 inch x1	Rp1,850,000
1003	Rafi Ardiansyah	Mouse Wireless x3	Rp525,000

One customer (Mafaza Husnadani) has two orders while another has one, which proves the multiplicity 1 to 0..*. Adding ten units of a product whose stock is only two is rejected by Stock.isAvailable(), which proves that the business rule is kept inside the Stock object.

PART 3 - QUESTION 2: Class Diagram for the "Bengkel Maju" System

An automotive workshop called "Bengkel Maju" wants to develop a simple system to manage customer vehicle data for vehicles brought in for repair. The Java-based system should be able to record vehicle information (cars and motorcycles), display service information, and calculate the estimated cost based on the type of vehicle and type of service.

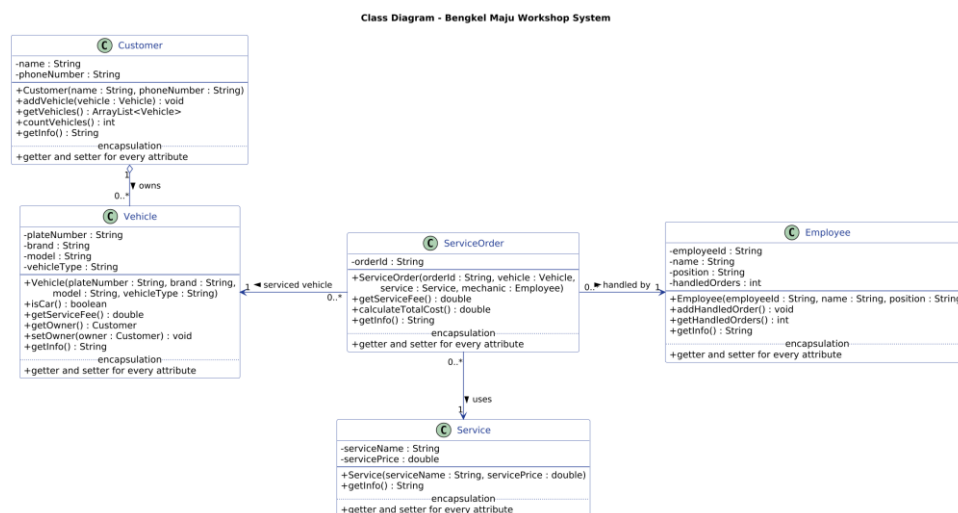
1. Analysis of the Requirements

The workshop needs to record customer vehicles, show the service taken by each vehicle, and estimate the total cost. Five classes are used:

- Customer - stores name and phoneNumber and owns the list of vehicles.
- Vehicle - stores plateNumber, brand, model, vehicleType, and knows its owner. It also decides its own service fee (Rp50,000 for a car, Rp20,000 for a motorcycle), because the fee depends only on the type of vehicle.
- Service - the workshop service catalogue: serviceName and servicePrice.
- Employee - the additional class required by the quiz. It represents the mechanic who handles the job, with employeeId, name, position, and a counter handledOrders.
- ServiceOrder - one service transaction. It connects a vehicle, the chosen service, and the mechanic in charge, and it calculates the total estimated cost.

ServiceOrder is added so that each class keeps a single responsibility. Without it, class Vehicle would have to store the chosen service, the assigned mechanic, and the transaction number, which are transaction data and not vehicle data. With ServiceOrder the same vehicle can also come back to the workshop several times, each visit producing its own record.

2. Class Diagram



3. Explanation of the Relationships

Relationship	Type of Relation	Multiplicity	Reason
Customer - Vehicle	Aggregation (hollow diamond on the Customer end)	1 to 0..*	A customer owns the vehicles, so it is a "has-a" whole-part relation. It is an aggregation and not a composition because a vehicle still exists even if the customer record is removed from the system - the vehicle has its own life cycle. One customer may bring many vehicles, and each vehicle

Relationship	Type of Relation	Multiplicity	Reason
			belongs to exactly one customer, exactly as required by the quiz.
ServiceOrder - Vehicle	Directed association	0..* to 1	One service order services exactly one vehicle, while one vehicle may be serviced many times.
ServiceOrder - Service	Directed association	0..* to 1	Each order uses one service from the catalogue; the same service (for example Ganti Oli) may be used by many orders.
ServiceOrder - Employee	Directed association	0..* to 1	Each order is handled by one mechanic, and one mechanic may handle many orders.

The association between Customer and Vehicle is bidirectional: Customer keeps an ArrayList of its vehicles, and Vehicle keeps a reference to its owner. The method `addVehicle()` sets both sides at once so that the two references can never contradict each other. The other three relationships are drawn with an open arrowhead because they are navigable in one direction only - a ServiceOrder knows its vehicle, service and mechanic, but those classes do not keep a list of orders.

4. Application of Encapsulation

- Every attribute in all five classes is declared private, so no attribute can be accessed directly from outside its class.
- Each class provides a constructor that initialises its attributes when the object is created.
- Access is given only through public getter and setter methods.
- The setters validate the incoming data where it matters, for example `setServicePrice()` rejects a negative price and `addVehicle()` ignores a null vehicle and prevents the same vehicle from being registered twice.
- The cost rule is hidden inside the objects: `Vehicle.getServiceFee()` knows the fee of its own type and `ServiceOrder.calculateTotalCost()` adds it to the service price, so the Main class never performs the calculation itself.

5. Cost Calculation Rule

Vehicle Type	Formula	Implementation
Car (Mobil)	<code>servicePrice + Rp50,000</code>	<code>Vehicle.getServiceFee()</code> returns 50000 when <code>isCar()</code> is true
Motorcycle (Motor)	<code>servicePrice + Rp20,000</code>	<code>Vehicle.getServiceFee()</code> returns 20000 otherwise

PART 4 - QUESTION 2: Java Implementation

The program consists of six Java files, stored in the folder Quiz/Soal2 and declared as package Soal2. From the folder Quiz the program is compiled with `javac Soal2/*.java` and executed with `java Soal2.BengkelMajuDemo14`.

1. Customer14.java

```
Customer14.java U X
Quiz > Customer14.java
1  import java.util.ArrayList;
2
3  public class Customer14 {
4      private String name;
5      private String phoneNumber;
6      private ArrayList<Vehicle14> vehicles;
7
8      public Customer14(String name, String phoneNumber) {
9          this.name = name;
10         this.phoneNumber = phoneNumber;
11         this.vehicles = new ArrayList<Vehicle14>();
12     }
13     Loading...
14     public String getName() {
15         return name;
16     }
17
18     public void setName(String name) {
19         this.name = name;
20     }
21
22     public String getPhoneNumber() {
23         return phoneNumber;
24     }
25
26     public void setPhoneNumber(String phoneNumber) {
27         this.phoneNumber = phoneNumber;
28     }
29
30     public ArrayList<Vehicle14> getVehicles() {
31         return vehicles;
32     }
33
34     public void addVehicle(Vehicle14 vehicle) {
35         if (vehicle == null) {
36             return;
37         }
38
39         if (!vehicles.contains(vehicle)) {
40             vehicles.add(vehicle);
41             vehicle.setOwner(this);
42         }
43     }
44
45     public int countVehicles() {
46         return vehicles.size();
47     }
48
49     public String getInfo() {
50         return name + " (" + phoneNumber + ") - "
51             + countVehicles() + " kendaraan";
52     }
53 }
54
```

2. Vehicle14.java

```
Vehicle14.java U X
Quiz > Vehicle14.java > Vehicle14
1 public class Vehicle14 {
2     public static final String TYPE_CAR = "Mobil";
3     public static final String TYPE_MOTORCYCLE = "Motor";
4
5     private String plateNumber;
6     private String brand;
7     private String model;
8     private String vehicleType;
9     private Customer14 owner;
10
11     public Vehicle14(String plateNumber, String brand, String model,
12         String vehicleType) {
13         this.plateNumber = plateNumber;
14         this.brand = brand;
15         this.model = model;
16         this.vehicleType = vehicleType;
17     }
18
19     public String getPlateNumber() {
20         return plateNumber;
21     }
22
23     public void setPlateNumber(String plateNumber) {
24         this.plateNumber = plateNumber;
25     }
26
27     public String getBrand() {
28         return brand;
29     }
30
31     public void setBrand(String brand) {
32         this.brand = brand;
33     }
34
35     public String getModel() {
36         return model;
37     }
38 }
```

```
Vehicle14.java U X
Quiz > Vehicle14.java > Vehicle14
1 public class Vehicle14 {
35     public String getModel() {
36         return model;
37     }
38
39     public void setModel(String model) {
40         this.model = model;
41     }
42
43     public String getVehicleType() {
44         return vehicleType;
45     }
46
47     public void setVehicleType(String vehicleType) {
48         this.vehicleType = vehicleType;
49     }
50
51     public Customer14 getOwner() {
52         return owner;
53     }
54
55     public void setOwner(Customer14 owner) {
56         this.owner = owner;
57     }
58
59     public boolean isCar() {
60         return TYPE_CAR.equalsIgnoreCase(vehicleType);
61     }
62
63     public double getServiceFee() {
64         if (isCar()) {
65             return 50000;
66         }
67
68         return 20000;
69     }
70 }
```

```
62
63     public double getServiceFee() {
64         if (isCar()) {
65             return 50000;
66         }
67
68         return 20000;
69     }
70
71     public String getInfo() {
72         return vehicleType + " " + brand + " " + model
73             + " (" + plateNumber + ")";
74     }
75 }
76 }
```

3. Service14.java

```
Service14.java U X
Quiz > Service14.java > Service14
1 public class Service14 {
2     private String serviceName;
3     private double servicePrice;
4
5     public Service14(String serviceName, double servicePrice) {
6         this.serviceName = serviceName;
7         this.servicePrice = servicePrice;
8     }
9
10    public String getServiceName() {
11        return serviceName;
12    }
13
14    public void setServiceName(String serviceName) {
15        this.serviceName = serviceName;
16    }
17
18    public double getServicePrice() {
19        return servicePrice;
20    }
21
22    public void setServicePrice(double servicePrice) {
23        if (servicePrice >= 0) {
24            this.servicePrice = servicePrice;
25        }
26    }
27
28    public String getInfo() {
29        return serviceName + " - Rp" + (long) servicePrice;
30    }
31 }
32
```

4. Employee14.java

```
Employee14.java U X
Quiz > Employee14.java > Employee14
1 public class Employee14 {
2     private String employeeId;
3     private String name;
4     private String position;
5     private int handledOrders;
6
7     public Employee14(String employeeId, String name, String position) {
8         this.employeeId = employeeId;
9         this.name = name;
10        this.position = position;
11        this.handledOrders = 0;
12    }
13
14    public String getEmployeeId() {
15        return employeeId;
16    }
17
18    public void setEmployeeId(String employeeId) {
19        this.employeeId = employeeId;
20    }
21
22    public String getName() {
23        return name;
24    }
25
26    public void setName(String name) {
27        this.name = name;
28    }
29
30    public String getPosition() {
31        return position;
32    }
33
34    public void setPosition(String position) {
35        this.position = position;
36    }
37 }
Ln 1, Col 1 Spaces: 4 UTF-8 LF ( ) Java
```

```

34     public void setPosition(String position) {
35         this.position = position;
36     }
37
38     public int getHandledOrders() {
39         return handledOrders;
40     }
41
42     public void addHandledOrder() {
43         this.handledOrders++;
44     }
45
46     public String getInfo() {
47         return name + " (" + employeeId + ") - " + position;
48     }
49 }
50

```

5. ServiceOrder14.java

```

ServiceOrder14.java U X
Quiz > ServiceOrder14.java > ServiceOrder14
1  public class ServiceOrder14 {
2      private String orderId;
3      private Vehicle14 vehicle;
4      private Service14 service;
5      private Employee14 mechanic;
6
7      public ServiceOrder14(String orderId, Vehicle14 vehicle,
8          Service14 service, Employee14 mechanic) {
9          this.orderId = orderId;
10         this.vehicle = vehicle;
11         this.service = service;
12         this.mechanic = mechanic;
13
14         if (mechanic != null) {
15             mechanic.addHandledOrder();
16         }
17     }
18
19     public String getOrderId() {
20         return orderId;
21     }
22
23     public void setOrderId(String orderId) {
24         this.orderId = orderId;
25     }
26
27     public Vehicle14 getVehicle() {
28         return vehicle;
29     }
30
31     public void setVehicle(Vehicle14 vehicle) {
32         this.vehicle = vehicle;
33     }
34
35     public Service14 getService() {
36         return service;
37     }

```

```

ServiceOrder14.java U X
Quiz > ServiceOrder14.java > ServiceOrder14
1 public class ServiceOrder14 {
2
39 public void setService(Service14 service) {
40     this.service = service;
41 }
42
43 public Employee14 getMechanic() {
44     return mechanic;
45 }
46
47 public void setMechanic(Employee14 mechanic) {
48     this.mechanic = mechanic;
49 }
50
51 public double getServiceFee() {
52     return vehicle.getServiceFee();
53 }
54
55 public double calculateTotalCost() {
56     return service.getServicePrice() + vehicle.getServiceFee();
57 }
58
59 public String getInfo() {
60     String info = "";
61     info += "No. Order      : " + orderId + "\n";
62     info += "Pelanggan       : " + vehicle.getOwner().getName() + "\n";
63     info += "No. Telepon        : " + vehicle.getOwner().getPhoneNumber() + "\n";
64     info += "Kendaraan         : " + vehicle.getInfo() + "\n";
65     info += "Servis Dipilih    : " + service.getServiceName() + "\n";
66     info += "Biaya Servis     : Rp" + (long) service.getServicePrice() + "\n";
67     info += "Biaya Jasa       : Rp" + (long) getServiceFee()
68         + " (" + vehicle.getVehicleType() + ") \n";
69     info += "Mekanik          : " + mechanic.getInfo() + "\n";
70     info += "TOTAL ESTIMASI  : Rp" + (long) calculateTotalCost() + "\n";
71
72     return info;
73 }

```

6. BengkelMajuDemo14.java (Main)

```

BengkelMajuDemo14.java U X
Quiz > BengkelMajuDemo14.java > ...
1 import java.util.ArrayList;
2
3 public class BengkelMajuDemo14 {
4     public static void main(String[] args) {
5         // 1. Data pelanggan
6         Customer14 mafaza = new Customer14(name: "Mafaza Husnadani", phoneNumber: "081234567890");
7         Customer14 rafi = new Customer14(name: "Rafi Ardiansyah", phoneNumber: "085612340987");
8
9         // 2. Data kendaraan (2 mobil + 2 motor)
10        Vehicle14 avanza = new Vehicle14(
11            plateNumber: "N 1234 AB", brand: "Toyota", model: "Avanza", Vehicle14.TYPE_CAR);
12        Vehicle14 beat = new Vehicle14(
13            plateNumber: "N 5678 CD", brand: "Honda", model: "Beat", Vehicle14.TYPE_MOTORCYCLE);
14        Vehicle14 brio = new Vehicle14(
15            plateNumber: "N 9012 EF", brand: "Honda", model: "Brio", Vehicle14.TYPE_CAR);
16        Vehicle14 vario = new Vehicle14(
17            plateNumber: "N 3456 GH", brand: "Honda", model: "Vario 160", Vehicle14.TYPE_MOTORCYCLE);
18
19        // Satu pelanggan boleh punya banyak kendaraan,
20        // satu kendaraan hanya milik satu pelanggan.
21        mafaza.addVehicle(avanza);
22        mafaza.addVehicle(beat);
23        mafaza.addVehicle(brio);
24        rafi.addVehicle(vario);
25
26        // 3. Daftar Layanan bengkel
27        Service14 gantiOli = new Service14(serviceName: "Ganti Oli Mesin", servicePrice: 120000);
28        Service14 tuneUp = new Service14(serviceName: "Tune Up Mesin", servicePrice: 250000);
29        Service14 servisRem = new Service14(serviceName: "Servis Rem", servicePrice: 90000);
30        Service14 servisRutin = new Service14(serviceName: "Servis Rutin", servicePrice: 75000);
31
32        // 4. Data karyawan
33        Employee14 budi = new Employee14(employeeId: "EMP01", name: "Budi Santoso", position: "Mekanik Mobil");
34        Employee14 sari = new Employee14(employeeId: "EMP02", name: "Sari Wulandari", position: "Mekanik Motor");
35
36        // 5. Transaksi servis

```

```

BengkelMajuDemo14.java U X
Quiz > BengkelMajuDemo14.java >...
3 public class BengkelMajuDemo14 {
4     public static void main(String[] args) {
5         // 5. Transaksi servis
6         ArrayList<ServiceOrder14> orders = new ArrayList<ServiceOrder14>();
7         orders.add(new ServiceOrder14(orderId: "SO001", avanza, tuneUp, budi));
8         orders.add(new ServiceOrder14(orderId: "SO002", beat, gantioli, sari));
9         orders.add(new ServiceOrder14(orderId: "SO003", brio, servisRem, budi));
10        orders.add(new ServiceOrder14(orderId: "SO004", vario, servisRutin, sari));
11
12        // 6. Tampilkan data pelanggan dan kendaraannya
13        System.out.println(x: "=====");
14        System.out.println(x: "        BENGKEL MAJU - DATA PELANGGAN        ");
15        System.out.println(x: "=====");
16
17        ArrayList<Customer14> customers = new ArrayList<Customer14>();
18        customers.add(mafaza);
19        customers.add(rafi);
20
21        for (Customer14 c : customers) {
22            System.out.println("Nama Pelanggan : " + c.getName());
23            System.out.println("No. Telepon   : " + c.getPhoneNumber());
24            System.out.println(x: "Daftar Kendaraan:");
25
26            for (Vehicle14 v : c.getVehicles()) {
27                System.out.println("    - " + v.getInfo());
28            }
29
30            System.out.println();
31        }
32
33        // 7. Tampilkan seluruh informasi servis
34        System.out.println(x: "=====");
35        System.out.println(x: "        DAFTAR ESTIMASI BIAYA SERVIS        ");
36        System.out.println(x: "=====");
37
38        double grandTotal = 0;

```

```

70
71        for (ServiceOrder14 order : orders) {
72            System.out.println(order.getInfo());
73            System.out.println(x: "-----");
74            grandTotal += order.calculateTotalCost();
75        }
76
77        System.out.println("TOTAL SELURUH ESTIMASI : Rp" + (long) grandTotal);
78        System.out.println();
79
80        // 8. Rekap pekerjaan mekanik
81        System.out.println(x: "=====");
82        System.out.println(x: "        REKAP PEKERJAAN MEKANIK        ");
83        System.out.println(x: "=====");
84        System.out.println(budi.getInfo() + " | Order ditangani: "
85            + budi.getHandledOrders());
86        System.out.println(sari.getInfo() + " | Order ditangani: "
87            + sari.getHandledOrders());
88    }
89 }
90

```

PART 5 - QUESTION 2: Program Output

```
Problems Output Debug Console Terminal Ports
PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+Show
CodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet\bin' 'Soal2.BengkelMaj
uDemo14'

=====
          BENGKEL MAJU - DATA PELANGGAN
=====

Nama Pelanggan : Mafaza Husnadani
No. Telepon    : 081234567890
Daftar Kendaraan:
- Mobil Toyota Avanza (N 1234 AB)
- Motor Honda Beat (N 5678 CD)
- Mobil Honda Brio (N 9012 EF)

Nama Pelanggan : Rafi Ardiansyah
No. Telepon    : 085612340987
Daftar Kendaraan:
- Motor Honda Vario 160 (N 3456 GH)

=====
          DAFTAR ESTIMASI BIAYA SERVIS
=====

No. Order      : S0001
Pelanggan      : Mafaza Husnadani
No. Telepon    : 081234567890
Kendaraan      : Mobil Toyota Avanza (N 1234 AB)
Servis Dipilih : Tune Up Mesin
Biaya Servis   : Rp250000
Biaya Jasa     : Rp50000 (Mobil)
Mekanik        : Budi Santoso (EMP01) - Mekanik Mobil
TOTAL ESTIMASI : Rp300000

-----

No. Order      : S0002
Pelanggan      : Mafaza Husnadani
No. Telepon    : 081234567890
Kendaraan      : Motor Honda Beat (N 5678 CD)
Servis Dipilih : Ganti Oli Mesin
Biaya Servis   : Rp120000
Biaya Jasa     : Rp20000 (Motor)
Mekanik        : Sari Wulandari (EMP02) - Mekanik Motor
TOTAL ESTIMASI : Rp140000
```

```
Problems Output Debug Console Terminal Ports

PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+Show
CodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet\bin' 'Soal2.BengkelMaj
uDemo14'

-----

No. Order      : S0002
Pelanggan      : Mafaza Husnadani
No. Telepon    : 081234567890
Kendaraan      : Motor Honda Beat (N 5678 CD)
Servis Dipilih : Ganti Oli Mesin
Biaya Servis   : Rp120000
Biaya Jasa     : Rp20000 (Motor)
Mekanik        : Sari Wulandari (EMP02) - Mekanik Motor
TOTAL ESTIMASI : Rp140000

-----

No. Order      : S0003
Pelanggan      : Mafaza Husnadani
No. Telepon    : 081234567890
Kendaraan      : Mobil Honda Brio (N 9012 EF)
Servis Dipilih : Servis Rem
Biaya Servis   : Rp90000
Biaya Jasa     : Rp50000 (Mobil)
Mekanik        : Budi Santoso (EMP01) - Mekanik Mobil
TOTAL ESTIMASI : Rp140000

-----

No. Order      : S0004
Pelanggan      : Rafi Ardiansyah
No. Telepon    : 085612340987
Kendaraan      : Motor Honda Vario 160 (N 3456 GH)
Servis Dipilih : Servis Rutin
Biaya Servis   : Rp75000
Biaya Jasa     : Rp20000 (Motor)
Mekanik        : Sari Wulandari (EMP02) - Mekanik Motor
TOTAL ESTIMASI : Rp95000

-----

TOTAL SELURUH ESTIMASI : Rp675000
```

```
-----  
TOTAL SELURUH ESTIMASI : Rp675000
```

```
=====
```

REKAP PEKERJAAN MEKANIK

```
=====
```

Budi Santoso (EMP01) - Mekanik Mobil | Order ditangani: 2

Sari Wulandari (EMP02) - Mekanik Motor | Order ditangani: 2

PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 3\Praktikum PBO\Jobsheet>

Verification of the Calculation

Order	Vehicle	Type	Service Price	Fee	Total
SO001	Toyota Avanza (N 1234 AB)	Mobil	Rp250,000	Rp50,000	Rp300,000
SO002	Honda Beat (N 5678 CD)	Motor	Rp120,000	Rp20,000	Rp140,000
SO003	Honda Brio (N 9012 EF)	Mobil	Rp90,000	Rp50,000	Rp140,000
SO004	Honda Vario 160 (N 3456 GH)	Motor	Rp75,000	Rp20,000	Rp95,000
				Grand Total	Rp675,000

The output shows two cars and two motorcycles as required. For every order the program prints the customer name, the vehicle information, the selected service, the service fee that depends on the vehicle type, the assigned mechanic, and the total estimated cost.